

Boletín 1 LMAR

Lenguajes de Marcas y Sistemas de Gestión de Información (LMSGI)

Juan José Marchena Rubio

Tarea 1

Bloque 1 — Cuestiones teóricas sobre lenguajes de marcas

1.1. ¿Qué es un lenguaje de marcas?

Un lenguaje de marcas (markup language) es un sistema de anotación textual que combina el contenido de un documento con un conjunto de marcas o etiquetas que aportan información adicional sobre su estructura, su presentación o su significado semántico. Las marcas no forman parte del contenido real del documento, sino que se intercalan con él para indicar cómo debe interpretarse, organizarse o mostrarse. Un ejemplo sencillo sería `<titulo>Mi documento</titulo>`, donde la etiqueta `<titulo>` indica que ese fragmento de texto cumple la función de título.

1.2. ¿Por qué surgen los lenguajes de marcas?

Surgen por la necesidad, heredada del mundo editorial, de indicar sobre un texto instrucciones de formato o estructura sin mezclarlas de forma ambigua con el contenido. Tradicionalmente, los correctores de pruebas anotaban a mano sobre los manuscritos instrucciones tipográficas (poner en negrita, cambiar de tamaño, iniciar un párrafo nuevo) para que el impresor supiera cómo componer el texto. Al trasladar los documentos al entorno digital surgió la misma necesidad: anotar el texto electrónico con instrucciones de formato o de estructura que pudieran ser interpretadas de manera automática por distintos programas. Esto permite además separar el contenido de la presentación, facilitar el intercambio de información entre sistemas distintos y automatizar el procesamiento de los documentos.

1.3. Un lenguaje de marcas, ¿es un lenguaje de programación? Justifica tu respuesta.

No, un lenguaje de marcas no es un lenguaje de programación. Un lenguaje de programación permite describir algoritmos: secuencias de instrucciones, estructuras de control (condicionales, bucles), variables y operaciones, es decir, permite indicar a la máquina qué acciones debe ejecutar y en qué orden. Un lenguaje de marcas, en cambio, no contiene lógica ni se ejecuta: únicamente describe la estructura, el formato o el significado de los datos de un documento mediante etiquetas. No incorpora estructuras de control ni es, en general, computacionalmente completo. Por ello, HTML o XML se consideran lenguajes descriptivos o declarativos, y no lenguajes de programación (aunque existen tecnologías derivadas, como XSLT, que sí incorporan cierta capacidad de programación sobre documentos XML).

1.4. Clasificación de los lenguajes de marcas. Ejemplos.

Los lenguajes de marcas se pueden clasificar atendiendo a distintos criterios:

- **Por su función u orientación:** de procedimiento (indican instrucciones concretas de formato que el procesador ejecuta en el orden en que aparecen; ej. RTF, LaTeX/TeX), descriptivos o de presentación (describen la estructura lógica del documento —títulos, párrafos, listas— sin fijar cómo debe mostrarse; ej. HTML) y de datos o semánticos (permiten crear etiquetas propias para describir el significado de la información; ej. XML).
- **Por la gramática/metalenguaje en que se basan:** basados en SGML (ej. HTML 4.01, DocBook) y basados en XML (ej. XHTML, RSS, SVG, MathML).

1.5. ¿Qué es el GML? ¿En qué ámbitos se usaba? ¿Y el SGML?

GML (Generalized Markup Language) fue un lenguaje de marcas desarrollado en 1969 por Charles Goldfarb, Edward Mosher y Raymond Lorie en IBM (las siglas GML corresponden, de hecho, a los apellidos de sus creadores). Se creó con el objetivo de separar el contenido de un

documento de las instrucciones de formato, y se empleó fundamentalmente en el ámbito de la edición y gestión de documentación técnica y legal dentro de la propia IBM.

SGML (Standard Generalized Markup Language) es la evolución de GML, publicada como estándar ISO en 1986. No es un lenguaje de marcas concreto, sino un metalenguaje: un conjunto de reglas que permite definir otros lenguajes de marcas a medida según las necesidades de cada organización. Se ha utilizado en la industria editorial, en la administración pública, en la industria aeronáutica y militar (documentación técnica) y es, además, la base de la que deriva HTML.

1.6. ¿Por qué el lenguaje de marcas HTML se popularizó tan rápidamente?

HTML se popularizó con enorme rapidez por varios motivos combinados:

- Su sencillez: un número reducido de etiquetas, fáciles de aprender incluso sin conocimientos previos de programación.
- Nació ligado al propio origen de la Web (Tim Berners-Lee, 1991), por lo que creció al mismo ritmo que la expansión de Internet.
- Los navegadores lo interpretaban de forma muy tolerante ante errores de sintaxis, mostrando la página aunque el código no fuera perfecto.
- Es abierto y gratuito: cualquier editor de texto plano permite crear un documento HTML.
- Permitía enlazar documentos entre sí mediante hipervínculos, algo revolucionario para la época.

1.7. ¿Qué ventajas ofrece HTML y en qué ámbito se usa?

Entre sus ventajas destacan: es multiplataforma (se interpreta igual en cualquier sistema operativo con un navegador), muy fácil de aprender, soportado de forma universal por todos los navegadores, permite incluir texto, imágenes, enlaces y elementos multimedia, y —combinado con CSS y JavaScript— posibilita interfaces atractivas e interactivas. Al ser texto plano, es ligero y fácil de generar, depurar y transmitir. Su ámbito de uso principal es la creación de páginas y aplicaciones web, aunque también se emplea en correos electrónicos con formato, documentación en línea e interfaces de aplicaciones híbridas.

1.8. ¿Por qué surge XML? ¿Qué nos permite hacer?

HTML dispone de un conjunto de etiquetas fijo y orientado casi exclusivamente a la presentación, por lo que no permite describir el significado de los datos ni crear etiquetas propias adaptadas a un dominio concreto. Para resolver esta limitación, el W3C publica en 1998 XML (eXtensible Markup Language) como una simplificación de SGML pensada para ser usada con facilidad tanto por personas como por máquinas. XML nos permite: definir lenguajes de marcas propios adaptados a cualquier ámbito (facturación, configuración de aplicaciones, datos científicos, etc.), estructurar y dar significado semántico a los datos, intercambiar información entre sistemas y plataformas heterogéneas de forma estandarizada, separar por completo los datos de su presentación, y validar la estructura de un documento mediante un DTD o un XML Schema.

1.9. Explica las principales características de un lenguaje de marcas.

- Separan el contenido de la forma o estructura del documento.
- Se basan en texto plano, por lo que son legibles tanto por personas como por máquinas.
- Utilizan etiquetas, normalmente delimitadas por los símbolos < y >, que pueden anidarse formando una estructura jerárquica en forma de árbol.

- Las etiquetas pueden llevar atributos que añaden información complementaria.
- Siguen una gramática o conjunto de reglas sintácticas (a veces formalizada en un DTD o un Schema) que determina qué etiquetas existen y cómo pueden combinarse.
- Son independientes de la plataforma y de la aplicación que los procesa.
- Un documento puede ser simplemente «bien formado» (cumple la sintaxis general) o además «válido» (cumple estrictamente su gramática).

1.10. ¿Qué es una gramática de un lenguaje de marcas?

Es el conjunto de reglas que define qué etiquetas existen en el lenguaje, qué atributos puede tener cada una, en qué orden y con qué nivel de anidamiento pueden aparecer, y qué tipo de contenido pueden llevar. La gramática es lo que permite comprobar si un documento concreto es válido, es decir, si cumple estrictamente esas reglas, más allá de estar simplemente bien formado sintácticamente. En XML, la gramática de un documento se expresa habitualmente mediante un DTD o un XML Schema.

1.11. Define los siguientes conceptos: DTD y XML Schema.

DTD (Document Type Definition): es el mecanismo original, heredado de SGML, para definir la gramática de un documento de marcas: qué elementos existen, en qué orden y con qué anidamiento pueden aparecer, y qué atributos admite cada uno. Se escribe con una sintaxis propia (no es XML), es un mecanismo relativamente sencillo pero limitado, ya que no permite definir tipos de datos concretos (números, fechas, patrones de texto, etc.).

XML Schema (XSD): es un mecanismo más moderno, definido por el W3C, que cumple el mismo propósito que el DTD pero escrito en sintaxis XML. Es mucho más potente y flexible: permite definir tipos de datos (enteros, fechas, cadenas, decimales...), establecer restricciones (rangos de valores, patrones, longitudes), trabajar con espacios de nombres (namespaces) y reutilizar definiciones entre distintos esquemas.

Bloque 2 — Procesador de textos vs. editor de texto

¿Qué ocurre al abrir el .odt con el Bloc de notas?

No se muestra el texto de forma legible. El Bloc de notas interpreta cualquier archivo como texto plano, pero un .odt no es texto plano: en realidad es un archivo comprimido en formato ZIP que contiene en su interior varios ficheros, entre ellos content.xml (el contenido en XML), styles.xml (los estilos), meta.xml (metadatos) y, en su caso, carpetas con imágenes u otros recursos. Al abrir ese ZIP directamente con el Bloc de notas, este muestra la secuencia de bytes «en crudo»: aparecerán símbolos extraños, caracteres no imprimibles y, como mucho, algún fragmento reconocible de nombres de archivo o de etiquetas XML entremezclados con contenido ilegible, porque el Bloc de notas no sabe descomprimir el ZIP ni interpretar su estructura interna.

¿Qué diferencia existe entre un procesador de textos y un editor de textos?

Un **editor de texto** (como el Bloc de notas o Notepad++) trabaja únicamente con texto plano: no permite aplicar formato enriquecido (negrita, cursiva, tamaños o tipos de letra, colores, tablas...) y guarda el archivo tal cual, carácter a carácter, sin metadatos ni estructura interna adicional. Un **procesador de textos** (como LibreOffice Writer o Microsoft Word) permite aplicar formato enriquecido al documento y organizarlo con estilos, imágenes, tablas, encabezados, etc. Para poder guardar toda esa información adicional, utiliza formatos de archivo estructurados —a menudo comprimidos o binarios, como .odt o .docx— que contienen, además del texto, la información de formato, estilo y metadatos del documento.

Bloque 3 — Páginas HTML: guardado y formatos

¿Por qué se crea una carpeta asociada al guardar la página HTML? ¿Qué contiene?

Una página HTML normalmente enlaza recursos externos que no forman parte del propio archivo .htm: imágenes, hojas de estilo CSS, scripts JavaScript, tipos de letra, etc. Cuando el navegador guarda la página completa (opción «Página web, completa»), guarda el documento HTML principal en el archivo .htm indicado y, además, crea junto a él una carpeta (normalmente con un nombre del tipo «miPagina_archivos» o similar) donde almacena copias locales de todos esos recursos externos. Así, al abrir después el .htm sin conexión a Internet, la página se puede visualizar con el mismo aspecto que tenía en el navegador, porque las imágenes, estilos y scripts se cargan desde esa carpeta local en lugar de desde el servidor original.

Dicha carpeta contiene, por tanto, las imágenes de la página, los archivos de hojas de estilo (.css), los scripts (.js) y, en general, cualquier otro recurso auxiliar que la página referenciaba.

Al abrir el archivo .htm con el Bloc de notas, ¿qué ocurre?

A diferencia del caso del .odt, en este caso el archivo sí se abre correctamente y se puede leer, porque un archivo .htm/.html es, por definición, texto plano. Lo que se muestra es el código fuente HTML de la página (etiquetas, atributos y texto) tal cual está escrito, pero sin «renderizar»: no se ve la página con su formato visual, sino el propio marcado que el navegador interpreta para generar ese aspecto.

¿Qué es un archivo mhtml?

MHTML (MIME HTML) es un formato que empaqueta en un único archivo tanto el código HTML de la página como todos los recursos asociados (imágenes, hojas de estilo, scripts...), codificados dentro del propio fichero mediante MIME. Es, por tanto, una alternativa al sistema de «archivo .htm + carpeta de recursos»: en lugar de generar dos elementos (el .htm y su carpeta), se obtiene un único archivo autocontenido con extensión .mhtml o .mht que representa la página completa.

Bloque 4 — Análisis de ficheros XML reales

A continuación se analizan dos ficheros XML de tipo y propósito distintos, representativos de usos muy habituales de este lenguaje: uno describe un catálogo de datos (una ficha de libro) y otro es un canal de sindicación de contenidos (un feed RSS), un formato ampliamente usado en la Web y basado en XML.

Fichero 1: ficha de libro (catálogo de datos)

```
<?xml version="1.0" encoding="UTF-8"?>
<libro>
  <titulo>Introduction to Programming</titulo>
  <autor>John Smith</autor>
  <isbn>978-3-16-148410-0</isbn>
  <editorial>Tech Books</editorial>
  <anio>2024</anio>
  <precio moneda="USD">29.99</precio>
  <categorias>
    <categoria>Programación</categoria>
    <categoria>Tecnología</categoria>
  </categorias>
</libro>
```

Elementos: libro (raíz), titulo, autor, isbn, editorial, anio, precio, categorias y categoria (que se repite dentro de categorias).

Etiquetas: cada elemento se delimita con su etiqueta de apertura y cierre, por ejemplo <titulo>...</titulo>.

Atributos: el elemento precio tiene el atributo moneda, que indica en qué divisa está expresado el valor numérico.

¿Qué almacena? Los datos catalográficos y comerciales de un libro: identificación, autoría, editorial, año de publicación, precio y las categorías temáticas a las que pertenece.

¿Se podría añadir información? Sí; al ser un formato extensible, se podrían añadir nuevos elementos (por ejemplo <idioma>, <numeroPaginas> o <formato>) o nuevos atributos, siempre que el documento resultante siga estando bien formado y, si existiera, respete la gramática (DTD o XML Schema) definida para este tipo de documentos.

Fichero 2: canal de sindicación RSS (feed)

```
<?xml version="1.0" encoding="UTF-8"?>
<rss version="2.0">
  <channel>
    <title>Blog de ejemplo</title>
    <link>https://www.ejemplo.com</link>
    <description>Novedades del sitio</description>
    <item>
      <title>Primer artículo</title>
      <link>https://www.ejemplo.com/articulo-1</link>
      <pubDate>Mon, 01 Sep 2026 10:00:00 GMT</pubDate>
    </item>
  </channel>
</rss>
```

Elementos: rss (raíz), channel, title, link, description, item y pubDate (item puede repetirse una vez por cada entrada del feed).

Etiquetas: por ejemplo <channel>...</channel> delimita toda la información del canal.

Atributos: el elemento rss lleva el atributo version, que indica la versión de la especificación RSS que se está utilizando (en este caso, 2.0).

¿Qué almacena? La información necesaria para que un lector de noticias o agregador RSS

muestre las últimas publicaciones de un sitio web: título y descripción del canal, y por cada entrada su título, enlace y fecha de publicación.

¿Se podría añadir información? Sí; se podrían añadir más elementos <item> a medida que se publican nuevas entradas, o enriquecer cada entrada con elementos adicionales previstos por la especificación RSS, como <description> o <author> dentro de cada item.

Bloque 5 — La gramática de HTML

5.1. Gramática de HTML según el W3C

Las primeras versiones de HTML (hasta HTML 4.01) heredaron de SGML el uso de un DTD (Document Type Definition) para definir su gramática: qué elementos existen, qué atributos admite cada uno y cómo pueden anidarse. De hecho, HTML 4.01 y XHTML 1.0 llegaron a publicar varios DTD alternativos (Strict, Transitional y Frameset), cada uno con un grado distinto de permisividad respecto a etiquetas de presentación ya consideradas obsoletas.

Con HTML5, el W3C (junto con el WHATWG) abandona el uso de un DTD formal. En su lugar, la gramática de HTML5 se define directamente en la propia especificación mediante una descripción detallada en prosa técnica del vocabulario permitido, unida a un algoritmo de análisis (parsing) muy concreto que indica exactamente cómo debe procesar un navegador cualquier documento, incluso si contiene errores de sintaxis. Esto supone un cambio de enfoque: en lugar de exigir que el documento sea estrictamente válido frente a un DTD, se prioriza que cualquier navegador procese el documento de forma predecible y uniforme, tolerando errores comunes de marcado.

5.2. Diferencia entre HTML y HTML5. Dos etiquetas de cada uno.

HTML (hasta la versión 4.01) está orientado principalmente a la presentación visual y basado en la gramática SGML: incluye numerosas etiquetas de formato directo, requiere el uso de plugins externos (como Flash o applets de Java) para incorporar contenido multimedia, y carece de etiquetas semánticas para describir la estructura del documento (secciones, cabeceras, artículos...).

HTML5, en cambio, introduce etiquetas semánticas que describen el significado y la función de cada parte del documento, incorpora de forma nativa la reproducción de audio y vídeo sin necesidad de plugins, añade nuevas capacidades gráficas y de interactividad (como el elemento canvas), nuevos tipos de campos de formulario y APIs adicionales (geolocalización, almacenamiento local, etc.), además de simplificar enormemente la declaración del tipo de documento (`<!DOCTYPE html>`).

Dos etiquetas típicas de HTML (de presentación, hoy obsoletas): `<font>` y `<center>`.

Dos etiquetas propias de HTML5 (semánticas/multimedia): `<article>` y `<video>`.

5.3. Etiqueta, elemento y atributo en HTML. Ejemplo.

Etiqueta (tag): es la marca delimitada por los símbolos `<` y `>` que señala el inicio o el final de un elemento. Por ejemplo, `<p>` es la etiqueta de apertura y `</p>` la etiqueta de cierre.

Elemento: es la unidad completa formada por la etiqueta de apertura, el contenido que encierra y su correspondiente etiqueta de cierre (o, en el caso de elementos vacíos, una única etiqueta autocontenida, como `<img />`). Por ejemplo, `<p>Hola mundo</p>` constituye un elemento de tipo párrafo completo.

Atributo: es información adicional que se incorpora dentro de la etiqueta de apertura, en forma de pares `nombre="valor"`, y que modifica o complementa las características del elemento. Por ejemplo, en `<p class="destacado">Hola mundo</p>`, `class="destacado"` es un atributo del elemento `p` que indica que debe aplicársele el estilo asociado a la clase «destacado».